

38 — Performance & Reliability Specification

Version 1.0 | Engineering Specification | September 2026

Performance must be optimized without weakening safety, verification, evidence, governance, or trusted-state integrity.

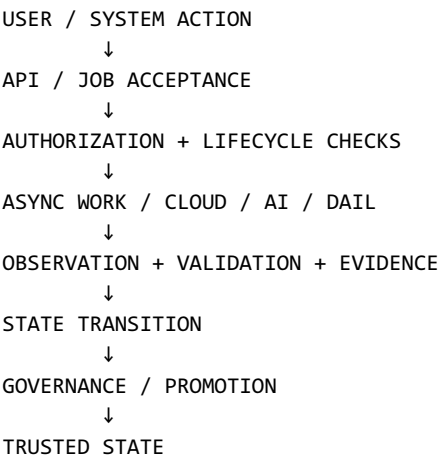
Document Control & Engineering Contract

Field	Value
Document	CloudShield AI Performance & Reliability Specification
Identifier	38
Version	1.0
Status	Implementation baseline
Scope	Entire CloudShield AI product, including DAIL as a bounded subsystem
Primary cloud	AWS for MVP
Architecture baseline	Modular application + asynchronous workers; small number of services where justified
System of record	Relational database; object storage for large evidence/artifacts
Core rule	Performance optimizations cannot bypass authorization, lifecycle, DAIL verification, governance, evidence, or trusted-state rules

Executive Summary

This specification defines the measurable performance, scalability, reliability, resilience, capacity, benchmarking, and recovery requirements for CloudShield AI. It converts the product's safety-oriented architecture into operational performance contracts. The system must remain responsive for interactive work, durable for asynchronous work, predictable under load, and safe under partial failure. CloudShield AI must prefer explicit degraded or unknown states over false confidence.

Canonical Performance/Reliability Contract



Performance may accelerate this path.
It may never remove or weaken a safety gate.

1. Purpose and Scope

This specification establishes the performance and reliability engineering contract for:

- Interactive web and API requests.

- Cloud discovery, configuration observation, normalization, and reconciliation.
- Security analysis, findings, risk scoring, dependency and impact analysis.
- AI/LLM requests, context assembly, validation, and degraded operation.
- DAIL verification, evidence generation, and governed promotion workflows.
- Database, queue, worker, object-storage, and eventing behavior.
- External integrations, notifications, webhooks, and asynchronous jobs.
- Multi-tenant isolation, capacity planning, load/stress/soak testing, and failure injection.
- Deployment, rollback, disaster recovery, and reconciliation.

Out of scope: changing the product's functional authority model, replacing DAIL verification with performance shortcuts, or treating benchmark success as authorization.

2. Performance & Reliability Objectives

Objective	Requirement
Responsiveness	Interactive reads should feel predictable; normal single-resource read APIs target p95 ≤ 500 ms.
Durability	Valid asynchronous jobs target ≥99.9% durable acceptance.
Availability	MVP API/control-plane availability target ≥99.5% monthly, excluding planned maintenance.
Safe completion	Every accepted critical promotion reaches a terminal outcome or explicit reconciliation state; no silent failure.
Audit durability	≥99% of critical audit/security events must be durably recorded.
Degradation	AI and external integrations may degrade independently without corrupting authoritative state.
Recovery	Critical API/control plane RTO ≤2h and RPO ≤15m; core DB RTO ≤1h and RPO ≤15m.
Isolation	One tenant's workload must not exhaust shared capacity or materially violate another tenant's service objectives.
Safety	Unknown, partial, stale, or ambiguous state must never be represented as clean/trusted merely to meet latency targets.

3. Performance Model

Performance is modeled as latency, throughput, concurrency, freshness, durability, and resource efficiency. Each workload must define its dominant dimension rather than optimizing a single global number.

Dimension	Meaning	Typical measurement
Latency	Time from request/job acceptance to defined milestone	p50/p95/p99
Throughput	Completed work per unit time	requests/s, assets/min, jobs/min
Concurrency	Simultaneous active work	in-flight requests/jobs
Freshness	Age of authoritative observation	seconds/minutes/hours
Durability	Probability accepted work/state	durable acceptance rate

	survives failure	
Efficiency	Resource consumed per unit of useful work	CPU, memory, DB I/O, tokens, cost
Tail behavior	Worst-case experience under contention	p95/p99/p99.9

3.1 Measurement Rules

- Report at least p50, p95, and p99 for latency-sensitive paths; averages alone are insufficient.
- Separate queue wait, execution time, dependency latency, and end-to-end latency.
- Measure cold and warm paths where caching or initialization changes behavior.
- Record workload size and tenant scope with every benchmark.
- Do not hide retries, reconciliation, or partial failures inside a single success metric.
- All benchmark results must identify software version, infrastructure shape, configuration, dataset, provider/model version where relevant, and test date.

4. Reliability Model

Reliability means the product continues to provide correct, authorized, observable, and recoverable behavior despite component, provider, network, data, deployment, or workload failures.

- Correctness over availability when the two conflict on safety-critical operations.
- Fail closed for promotion, authorization, verification, and trusted-state transitions when required evidence or preconditions are unavailable.
- Fail soft for non-authoritative intelligence such as optional AI explanation, enrichment, and best-effort external notifications.
- Persist state before acknowledging operations that the product promises to recover.
- Use idempotency and reconciliation for retries and ambiguous external outcomes.
- Preserve evidence and audit trails during normal operation and recovery.
- Bound retries and concurrency so failures do not create retry storms.

4.1 Reliability Failure Classes

Class	Examples	Required behavior
Transient	Timeout, throttling, temporary 5xx	Bounded retry with jitter; preserve idempotency
Dependency	DB unavailable, queue unavailable, LLM unavailable	Degrade or fail closed according to authority
Data	Stale/partial/inconsistent observation	Mark state explicitly; do not infer clean state
Concurrency	Duplicate job, conflicting candidate update	Reject/serialize/reconcile deterministically
External ambiguity	Mutation timed out after possible execution	Enter reconciliation; do not blindly retry mutation
Capacity	Queue growth, CPU/DB saturation	Backpressure, admission control, scaling
Deployment	Bad release, migration issue	Stop promotion; rollback/recover; reconcile state

5. Workload Taxonomy

Workload	Class	Primary constraint	Typical execution
Single-resource reads	Interactive	Latency	Synchronous
Search/list/filter	Interactive/query	Latency + DB load	Synchronous with pagination
Cloud discovery	Batch	Provider rate limits + throughput	Async workers
Security analysis	Batch/compute	CPU + data volume	Async
Dependency graph build	Batch/graph	DB I/O + graph traversal	Async
Impact analysis	Critical async	Correctness + graph traversal	Async, version-bound
AI explanation	Interactive/async	Model latency + cost	Async or bounded synchronous
DAIL verification	Critical async	Deterministic checks + cloud/provider calls	Async
Promotion	Critical control	Safety + external mutation	Governed async
Evidence/report generation	Batch	Object storage + compute	Async
Webhook delivery	Integration	External latency/rate limits	Async
Drift monitoring	Scheduled	Freshness + provider quotas	Async

6. Performance SLI / SLO Definitions

SLI	Definition	MVP target / baseline
API availability	Successful authorized requests / eligible requests	≥99.5% monthly
Read latency	End-to-end normal single-resource GET p95	≤500 ms
Async acceptance	Valid jobs durably accepted within acceptance budget	p95 ≤1 s for mutating async acceptance
Job durability	Valid jobs durably recorded / valid submissions	≥99.9%
Critical completion	Accepted promotion jobs reaching terminal/reconciliation state	100% target; no silent failure
Audit durability	Critical audit/security events durably stored / expected events	≥99%
Discovery freshness	Age of latest authoritative observation	Per connection/resource-class policy
Queue delay	Time from enqueue to worker start	Workload-specific; continuously tracked
AI latency	LLM request start to validated response	Workload/model-specific benchmark
Verification latency	Candidate version ready to terminal verification result	Workload-specific benchmark

Evidence latency	Required evidence durable after operation milestone	Workload-specific; no promotion without required evidence
------------------	---	---

These are engineering baselines for MVP and must be validated under representative workload. They are not guarantees of provider behavior.

7. Reliability SLI / SLO Definitions

SLI	Measurement	Target / policy
Error-free critical transitions	Critical state transitions without invariant violation	100% safety target
Reconciliation completion	Ambiguous external operations entering resolved or explicitly unresolved state	100% tracked
Recovery point	Maximum accepted data loss after incident	≤15 min for critical MVP stores
Recovery time	Time to restore defined service tier	See DR targets
Idempotent duplicate safety	Duplicate commands causing unintended duplicate mutation	0 tolerated
Tenant isolation incidents	Cross-tenant data/control leakage	0 tolerated
Silent failures	Accepted critical operation with neither terminal state nor reconciliation	0 tolerated
Backup restore success	Successful restore drills against defined recovery artifacts	100% for critical recovery set

8. Capacity Model

Capacity planning must model the product as a set of coupled queues and bottlenecks. The limiting resource may be database connections, CPU, memory, cloud API quotas, LLM tokens, object storage throughput, queue workers, or human approval throughput.

```
Capacity envelope =
min(application capacity,
    database capacity,
    queue/worker capacity,
    cloud-provider quota,
    LLM/provider quota,
    object-storage throughput,
    integration quota,
    governance/approval capacity)
```

For each resource:
utilization, saturation point, safe operating threshold,
scale-out unit, recovery behavior, and cost/unit must be documented.

- Maintain headroom for burst traffic and recovery; do not plan normal operation at saturation.
- Separate interactive and batch capacity where contention could harm user-facing SLOs.
- Model per-tenant quotas and global quotas independently.
- Capacity models must include retry amplification and recovery workloads.

- Promotion/verification capacity must be sized for worst-case reconciliation, not only happy-path throughput.

9. API Performance

- Use bounded request payloads, pagination, selective projections, indexed filters, and request-level timeouts.
- Normal reads target p95 ≤ 500 ms; p99 must be monitored even when p95 passes.
- Mutating operations that launch durable work should return an accepted/job reference after durable acceptance rather than hold the connection for long work.
- Do not perform unbounded cloud discovery, LLM calls, graph traversal, or verification synchronously in an interactive request.
- Rate limits, authorization checks, and tenant filters must execute without bypass under performance pressure.
- Cache only data whose freshness and authorization semantics are explicit.

10. Web/UI Performance

- Dashboard and inventory screens must progressively load summary data before expensive secondary views.
- Use server-side pagination/filtering for large inventories; never fetch an entire tenant inventory merely to render a page.
- Dependency graphs should support bounded views, progressive expansion, and server-side traversal limits.
- AI-generated content should display explicit loading, stale, unavailable, or degraded states.
- Long-running operations must expose job status rather than block the UI.

11. Discovery Performance

- Discovery is asynchronous and partitionable by account, region, service, or resource class.
- Provider pagination and API throttling must be respected; throughput must not be increased by violating provider quotas.
- Use checkpoints so a failed partition can resume without repeating all work.
- Normalize and reconcile incrementally where safe; preserve observation provenance and timestamps.
- Discovery freshness is policy-driven by resource class rather than one universal target.

12. Security Analysis Performance

- Security rules should execute deterministically and in bounded batches.
- Pre-index frequently evaluated configuration attributes.
- Parallelize independent rules while preserving deterministic finding identity and evidence.
- Do not suppress findings solely because a dependency or observation is stale; surface the uncertainty.
- Separate expensive enrichment from the core finding decision where possible.

13. Risk Analysis Performance

- Risk scoring should be deterministic and cacheable when its input versions are unchanged.
- Invalidate risk assessments when material findings, dependencies, asset context, or policy versions change.
- Risk calculations must remain traceable to the exact inputs used.
- Batch recomputation should be incremental rather than tenant-wide unless explicitly required.

14. Dependency Graph Performance

- Canonical dependency data remains a domain model; query/read representations may be optimized independently.
- Use bounded traversals, depth limits, indexes, adjacency-oriented access patterns, and incremental graph updates.
- Distinguish direct and transitive impact to avoid unnecessary traversal.
- Cache derived graph views only with explicit invalidation/version semantics.
- Unknown or low-confidence edges must not be silently converted into bounded blast radius.

15. Impact Analysis Performance

- Impact analysis is candidate-version-bound and should reuse stable dependency graph state when still valid.
- Compute direct impact first and expand transitively only as policy requires.
- Parallelize independent dependency branches while preserving deterministic aggregation.
- Record graph/version/freshness inputs so a fast result remains auditable.
- Impact analysis may fail closed for critical promotion if required dependency data is unavailable.

16. AI/LLM Performance

- Use model routing by task class, latency requirement, quality threshold, and cost policy.
- Separate context retrieval/assembly latency from model inference latency.
- Bound prompt/context size and token budgets; avoid sending unnecessary tenant data.
- Cache only deterministic, authorization-safe, version-compatible results.
- Apply timeouts, retries, circuit breakers, and fallback providers/models where allowed.
- AI is best-effort/degraded and never authoritative; provider slowness must not block unrelated deterministic workflows.

17. DAIL Verification Performance

- Verification runs must be asynchronous and observable.
- Parallelize independent deterministic checks and cloud reads where it preserves correctness.
- Verification results must remain bound to exact candidate version and required evidence.
- Never skip or weaken a verification gate to meet latency.
- Cache only immutable or version-bound verification inputs; invalidate when candidate, dependencies, policy, or authoritative observations change.

18. Remediation/Promotion Performance

- Candidate creation may be fast, but promotion is governed control-plane work and must remain correctness-first.
- Preflight validation should fail quickly when authorization, freshness, dependencies, verification, approval, or policy preconditions are missing.
- External mutations must be idempotent where provider semantics permit and reconciled when outcomes are ambiguous.
- Promotion throughput must never be increased by parallelizing conflicting mutations without explicit dependency-safe scheduling.

19. Evidence/Audit Performance

- Critical evidence and audit records must be durably written before their dependent state transition is considered complete.
- Use asynchronous enrichment for non-critical metadata, but never defer required proof past a gate that depends on it.
- Batch archival operations where safe without weakening retention or integrity.
- Audit/event pipelines must expose lag and durability health.

20. Database Performance

- Use appropriate composite indexes for tenant, scope, lifecycle status, timestamps, stable resource identity, and job state.
- Keep transactions short and explicit; avoid holding database transactions across network calls.
- Use optimistic concurrency/version checks for mutable domain objects.
- Use connection pooling with bounded pool sizes and workload-aware limits.
- Partition or archive high-volume event/evidence metadata when growth requires it.
- Measure query p95/p99, lock waits, connection saturation, I/O, cache hit rate, and slow-query frequency.

21. Queue / Async Worker Performance

- Every accepted job has a durable identity and idempotency strategy.
- Queues must expose depth, age of oldest message, processing rate, retry count, dead-letter count, and worker utilization.
- Use bounded concurrency per workload and provider.
- Backpressure must prevent queue growth from exhausting databases, provider quotas, or memory.
- Dead-lettered critical work requires operational reconciliation, not silent disappearance.
- Worker scaling should consider both queue depth and work age.

22. Cloud API Throttling & Rate Limits

- Treat provider throttling as a normal operating condition.
- Use per-provider, per-account, and per-region concurrency/rate controls where appropriate.
- Retry with exponential backoff and jitter only for retryable failures.
- Do not retry non-idempotent mutations blindly after ambiguous outcomes.
- Track quota utilization and throttling rate as capacity indicators.
- Provider limits are external constraints and cannot be 'optimized away' by unsafe parallelism.

23. External Integration Performance

- External integrations are asynchronous by default.
- Webhook delivery uses bounded retries, exponential backoff, deduplication, and dead-letter handling.
- One slow external system must not block the core control plane.
- Integration-specific quotas and latency SLOs are tracked independently.
- External content entering AI workflows passes the same trust and injection defenses as other untrusted content.

24. Caching

- Cache keys must include tenant/scope and relevant semantic version inputs.
- Every cache has an explicit freshness/invalidation policy.
- Never cache authorization decisions across incompatible identity or policy versions.
- Do not cache mutable cloud state as trusted state.
- Negative caching must be bounded to avoid hiding newly available resources.
- Cache failure must degrade to authoritative retrieval, not fabricate data.

25. Concurrency

- Use idempotency keys for externally retried commands.
- Use optimistic locking/version numbers for candidate, policy, connection, and other mutable aggregates.
- Serialize conflicting promotions and mutations.
- Prevent duplicate job execution from producing duplicate side effects.
- Approval and verification must be invalidated when protected inputs change.
- Race conditions must resolve to an explicit state, never an implicit assumption.

26. Scalability

- Scale stateless API capacity horizontally.
- Scale discovery/security/AI/DAIL workers independently.
- Use workload-specific worker pools where one class can starve another.
- Scale reads with indexing, pagination, read optimization, and safe caching before adding architectural complexity.
- Scale database capacity vertically first where appropriate, then partition/shard only when justified by measured limits.
- Preserve canonical domain semantics while allowing derived read models later.

27. Load Testing

- Establish representative tenant sizes, asset counts, finding counts, graph densities, candidate counts, and job mixes.
- Test normal load, expected peak load, burst load, and recovery load.
- Include provider latency and throttling simulations.
- Validate p50/p95/p99, throughput, saturation, queue age, error rate, and cost.
- Run load tests against release candidates before material production-scale changes.

28. Stress Testing

- Push selected components beyond expected capacity to identify failure thresholds.
- Measure graceful degradation rather than only maximum throughput.
- Verify that overload does not bypass authorization, evidence, verification, or tenant isolation.
- Test database saturation, queue overload, worker exhaustion, provider throttling, LLM unavailability, and object-storage pressure.
- Document the first failing resource, failure mode, recovery path, and safe operating threshold.

29. Soak / Endurance Testing

- Run representative workloads for extended periods to detect memory leaks, connection leaks, queue drift, cache growth, log/telemetry amplification, and data accumulation.
- Include scheduled discovery, drift monitoring, AI requests, verification, and external webhook traffic.
- Verify stable tail latency and resource utilization over time.
- Perform checkpointed evidence and state-integrity validation during the run.

30. Fault Injection

- Inject network timeouts, 5xx responses, throttling, DNS failures, dependency unavailability, worker crashes, duplicate deliveries, delayed messages, and ambiguous mutation outcomes.
- Inject database failover and queue interruption.
- Simulate LLM provider failure and malformed model responses.
- Simulate stale/partial dependency observations.
- Validate that critical workflows fail closed and enter reconciliation where required.
- Every injected failure must have an expected safety outcome and recovery assertion.

31. Reliability Patterns

- Timeouts at every network boundary.
- Exponential backoff with jitter for retryable transient failures.
- Circuit breakers for unstable external dependencies.
- Bulkheads between interactive, batch, AI, and control workloads.
- Idempotency for retried commands and jobs.
- Dead-letter queues with explicit operational ownership.
- Checkpointing for long-running discovery and analysis.
- Reconciliation loops for ambiguous external outcomes.
- Health/readiness probes that reflect dependency-criticality appropriately.
- Graceful shutdown with in-flight job handling.

32. Degradation & Backpressure

- AI unavailable → deterministic product features continue; AI-dependent explanation is marked unavailable/degraded.
- External integration unavailable → core CloudShield state continues; delivery is retried and tracked.
- Cloud read unavailable → affected observations become stale/unknown; critical promotion may block.
- Dependency graph incomplete → impact may be incomplete/unknown; never report a falsely bounded blast radius.
- Queue pressure → reduce admission, prioritize critical work, and preserve interactive capacity.
- Database pressure → reject/defer non-critical work before critical state integrity is compromised.
- Backpressure decisions must be observable and tenant-aware.

33. Recovery & Reconciliation

- Recovery follows dependency order: identity/secrets → database/object storage → queues → application/workers → integrations → authoritative cloud reconciliation.
- Restored state is not automatically trusted state.
- Reconcile cloud observations after infrastructure or database recovery before declaring trusted state current.
- Re-run or invalidate DAIL verification where its evidence or inputs are no longer valid.

- Ambiguous mutations must be reconciled against authoritative provider state before retrying.
- Recovery evidence and audit records must be retained.

34. Multi-Tenant Performance Isolation

- Enforce per-tenant quotas for API, jobs, discovery, AI, and integrations where needed.
- Use fair scheduling or weighted queues for shared workers.
- Prevent noisy-neighbor effects through concurrency and resource limits.
- Tenant-specific performance telemetry must not expose another tenant's data.
- Critical control operations receive reserved capacity or priority where justified.
- Isolation failures are security incidents as well as reliability defects.

35. Cost / Performance Optimization

- Optimize total cost per useful unit of work, not raw throughput alone.
- Track compute, database, storage, network, cloud API usage, LLM tokens, and integration costs.
- Prefer batching, caching, incremental computation, right-sizing, and workload-specific scaling where semantics permit.
- LLM model selection must consider quality, safety, latency, and cost together.
- Do not trade away evidence, verification, governance, or authoritative confirmation for lower cost.

36. Performance Observability

- Emit request, job, tenant, component, provider, model, candidate-version, and verification identifiers where appropriate.
- Measure queue wait separately from execution time.
- Dashboards should expose p50/p95/p99 latency, throughput, saturation, error rate, retries, queue age, throttling, cache hit rate, and resource utilization.
- Alert on SLO burn, saturation, queue growth, retry storms, dead-letter growth, and critical reconciliation backlog.
- Use the observability contract from Specification 31; logs are not substitutes for audit/evidence.

37. Capacity Planning

- Review capacity at release milestones and on a recurring operational cadence.
- Maintain a forecast based on tenants, assets, discovery frequency, findings, graph edges, AI calls, verification runs, and integrations.
- Define scale-out thresholds before saturation.
- Reserve recovery capacity so disaster recovery does not compete blindly with normal workloads.
- Record assumptions and update forecasts from production telemetry.

38. Benchmark Harness

- Provide repeatable benchmark scenarios for API, discovery, analysis, graph traversal, impact, AI, DAIL verification, evidence, and end-to-end promotion.
- Support fixed datasets plus parameterized workload generators.
- Capture environment, release, infrastructure shape, provider conditions, model configuration, and dataset versions.

- Produce latency distributions, throughput, errors, resource utilization, queue metrics, cost/unit, and safety-gate outcomes.
- Compare baseline vs candidate versions and fail CI/release gates on defined regressions.
- Benchmark results become evidence linked to the tested software/configuration version.

39. MVP Performance Baseline

The MVP baseline is intentionally conservative. It establishes measurable engineering targets while allowing empirical tuning as real workload data becomes available.

Area	MVP baseline
API availability	≥99.5% monthly excluding planned maintenance
Normal single-resource reads	p95 ≤500 ms
Async mutation acceptance	p95 ≤1 s to durable acceptance
Valid job durability	≥99.9%
Critical promotion completion	No silent failures; terminal or reconciliation state required
Critical audit/security events	≥99% durable recording
Critical API/control-plane DR	RTO ≤2h; RPO ≤15m
Core database DR	RTO ≤1h; RPO ≤15m
Evidence/object storage DR	RTO ≤2h; RPO ≤15m
Async job DR	RTO ≤2h; RPO ≤15m
DAIL state DR	RTO ≤2h; RPO ≤15m
AI availability	Best-effort/degraded; never authoritative
External integrations	Best-effort/integration-specific
Unknown/partial/stale state	Explicitly surfaced; never treated as clean

39.1 MVP Workload Profiles

Profile	Representative scale variable	Required benchmark
Small tenant	Low thousands of assets	Interactive API + scheduled discovery
Medium tenant	Tens of thousands of assets	Discovery + security + graph + AI mix
Large tenant	High tens/hundreds of thousands where supported	Partitioned async workloads + tenant isolation
Recovery	Normal workload + restore/reconciliation	RTO/RPO + queue replay + authoritative reconciliation

Exact asset-count thresholds should be finalized after representative AWS account data is collected; this specification does not invent a production capacity guarantee.

40. Implementation Sequence

1. Instrument all latency, queue, dependency, resource-utilization, retry, and state-transition measurements.
2. Implement API request budgets, database indexes, pagination, bounded payloads, and async job acceptance.
3. Implement queue metrics, worker concurrency limits, retry/backoff, dead-letter handling, and checkpointing.
4. Establish provider rate limiting, cloud-call telemetry, and reconciliation mechanisms.

5. Implement baseline caching with explicit invalidation and freshness semantics.
6. Implement workload isolation for interactive, batch, AI, and DAIL workers.
7. Build representative performance datasets and benchmark harness.
8. Run load, stress, soak, and fault-injection suites against the MVP architecture.
9. Tune database, worker, cloud, AI, and object-storage capacity from measured bottlenecks.
10. Validate DR and recovery workloads under concurrent normal load.
11. Define production alert thresholds and capacity dashboards.
12. Re-baseline SLOs only from evidence; document any target change through engineering governance.

41. Security & Functional Acceptance Criteria

AC-01: Normal single-resource read API p95 is ≤ 500 ms under the defined representative MVP workload.

AC-02: Valid asynchronous mutations reach durable acceptance within $p95 \leq 1$ s under the defined baseline load.

AC-03: $\geq 99.9\%$ of valid submitted jobs are durably recorded in the benchmark/reliability test.

AC-04: No accepted critical promotion can terminate without a terminal state or explicit reconciliation state.

AC-05: Performance tests demonstrate that tenant isolation is preserved under noisy-neighbor workloads.

AC-06: Provider throttling produces bounded retry/backoff behavior and does not create uncontrolled retry amplification.

AC-07: Ambiguous external mutation outcomes enter reconciliation rather than blind duplicate execution.

AC-08: AI/LLM outage does not create trusted state, bypass deterministic verification, or block unrelated core workflows.

AC-09: Verification remains bound to the exact candidate version; performance optimizations do not permit stale verification reuse.

AC-10: Unknown, partial, or stale dependency/configuration state is never represented as clean solely to meet latency targets.

AC-11: Load/stress testing identifies component saturation thresholds and demonstrates graceful degradation.

AC-12: Soak testing demonstrates no unacceptable memory, connection, queue, cache, or evidence accumulation.

AC-13: Critical audit/security events meet $\geq 99\%$ durable-recording baseline during representative load and failure tests.

AC-14: DR tests meet MVP RTO/RPO targets for the defined recovery tiers or produce an explicit engineering gap with remediation plan.

AC-15: Benchmark results are reproducible and linked to software/configuration/dataset versions and safety-gate outcomes.

42. Traceability

Upstream specification	Performance/Reliability dependency
00 Master System Specification	Product boundary, control loop, trusted-state model
01 SRS	Availability, performance, reliability, data, API, security requirements
02 System Architecture	Deployment topology, workers, data stores, async architecture
03 Functional Specification	Workflows, lifecycle, async behavior, failure states
04 Product & UX	Interactive performance and degraded UX
05 Domain & Data Model	Canonical data, identity, evidence, versioning
06 State & Lifecycle	State transitions, invalidation, reconciliation

07 Identity & Dependency	Stable identity, graph semantics, impact correctness
08 Security & Threat Model	Trust boundaries, least privilege, threat controls
09 Intelligence & AI Architecture	AI abstraction, context, routing, evaluation
10 LLM Integration	Provider adapters, retries, timeouts, cost, telemetry
11 AI Safety & Guardrails	Safety gates, non-authority, safe degradation
12–26 DAIL subsystem package	DAIL verification, evidence, experiments, deployment, lifecycle
27 AWS & Terraform	AWS topology, infrastructure capacity, IaC, DR
28 API Specification	API contracts, pagination, idempotency, rate limits
29 Cloud Integration	Provider throttling, discovery, mutation, reconciliation
30 External Integration	Webhook/integration performance and isolation
31 Observability & Logging	SLIs, telemetry, alerts, audit/evidence
32 Security Operations	Detection, response, security telemetry
33 Disaster Recovery	RTO/RPO, restore, reconciliation
34 Deployment & DevOps	Release performance, deployment, rollback, recovery
35 System Testing	Load, resilience, integration, E2E and regression strategy
36 DAIL Experiment	Performance/reliability experiments within DAIL governance
37 AI Evaluation & Benchmark	AI quality, latency, reliability, cost and regression evaluation

42.1 Core Invariant

Performance engineering is subordinate to the product's trust model. No latency, throughput, cost, or availability optimization may produce an invalid state, bypass an authorization boundary, weaken DAIL verification, suppress required evidence, or manufacture trusted state.

43. Engineering Decisions

Decision	Rationale
Async by default for long-running work	Protect interactive responsiveness and isolate provider/compute latency.
Modular MVP architecture	Allows measured scaling before introducing unnecessary distributed-system complexity.
Relational canonical store	Provides transactional lifecycle, identity, audit, and evidence metadata semantics.
Queue-backed workers	Provides backpressure, retry, partitioning, and independent scaling.
Provider-aware throttling	Cloud/LLM/external quotas are real capacity boundaries.
Version-bound caching	Prevents stale or unauthorized reuse.
Fail closed on critical control paths	Safety correctness is more important than availability.
Best-effort AI/external integrations	Prevents non-authoritative dependencies from becoming single points of control.
Explicit reconciliation	External systems can return ambiguous outcomes; safe

	recovery requires authoritative confirmation.
Evidence-linked benchmarks	Performance claims must be reproducible and traceable.

44. Definition of Done

- Performance and reliability requirements are implemented for all MVP critical paths.
- API, worker, database, cloud, AI, DAIL, evidence, and integration telemetry is available.
- SLO/SLI calculations are automated and visible to operators.
- Representative load, stress, soak, and fault-injection suites exist and run in CI/CD or controlled environments as appropriate.
- Critical promotion has deterministic terminal/reconciliation behavior under failure.
- Tenant isolation is verified under contention.
- Cloud throttling, LLM failure, integration failure, and dependency failure are tested.
- Caching has explicit freshness/invalidation semantics.
- Capacity thresholds and scale-out policies are documented.
- MVP RTO/RPO targets are validated through recovery testing.
- Benchmark outputs are versioned, reproducible, and retained as engineering evidence.
- No optimization weakens authorization, verification, governance, evidence, or trusted-state semantics.
- All acceptance criteria in Section 41 are passed or have an explicitly governed exception.

45. Glossary

Term	Definition
SLO	Service Level Objective: target level of service performance/reliability.
SLI	Service Level Indicator: measured quantity used to evaluate an SLO.
p95 / p99	95th / 99th percentile latency; tail behavior metrics.
RTO	Recovery Time Objective: target maximum restoration time.
RPO	Recovery Point Objective: target maximum tolerable data-loss window.
Backpressure	Mechanism that limits admission/work when downstream capacity is constrained.
Idempotency	Property allowing safe repetition of an operation without unintended duplicate effect.
Reconciliation	Process of resolving application state against authoritative external state after ambiguity or recovery.
Trusted State	State earned only after required verification, governance, promotion, and authoritative confirmation.
DAIL	CloudShield AI's bounded verification/decision-support subsystem; not the whole product.
Tail latency	High-percentile latency experienced by the slowest requests.
Soak test	Extended-duration test for stability and resource leakage.
Fault injection	Controlled introduction of failures to validate resilience

	and recovery behavior.
Durable acceptance	Point at which accepted work is persisted sufficiently to survive defined failure modes.
Noisy neighbor	One tenant/workload consuming shared resources enough to degrade others.

CloudShield AI — Performance & Reliability Specification v1.0

Engineering baseline • Performance never outranks safety and trust